

The memory, skill & audit *layer* for any AI coding agent.

Read this before the pitch. **This is the document we study.** re-forge is not another copilot. It is the model-agnostic and tool-agnostic layer that makes whichever assistant a team uses accumulate institutional knowledge, forge personalized skills, and pass adversarial quality gates before anything ships. **Pre-revenue. No fabricated traction.** Every figure here is sourced.

THE 30-SECOND ELEVATOR PITCH

Your team runs Claude Code, Cursor, Codex and ChatGPT side by side, and **every one of them is an amnesiac**: stateless across sessions, ignorant of each other. *re-forge is the shared brain over all of them.*

It sits **above** the assistant as an operating procedure: specialist agents **attack each other's work** (a skeptic kills weak reasoning, an adversary kills weak sources), **write evidence to disk** you can audit, and **compound every lesson into a memory the team owns**. The hard part ships today on Claude Code with a 685-check green CI; the always-on cross-tool monitor is the roadmap on a foundation that already works.

The market just priced memory as infrastructure (Mem0 raised \$24M; AWS named it the exclusive memory provider for its Agent SDK). We capture that value vertically, for software teams, with a per-team data asset that gets stickier every week.

The one-liner (pick by room)

“ *The memory and skill layer that makes any AI coding agent remember, improve, and prove its work.*

“ Your team's coding agents are amnesiacs. re-forge is the shared brain.

“ Model-agnostic, tool-agnostic. We win no matter which assistant wins.

“ Observability tells you what broke. re-forge stops it breaking and remembers why.

WHAT'S INSIDE • TAP TO JUMP

01 The market (TAM / SAM / SOM) & why now

02 Competitors & why we win (4 categories)

03 The moat (3 layers, honest)

04 Pricing & traction / proof

05 Technical summary (condensed)

06 Investor objections + rebuttals

07 Likely technical questions + answers

08 Demo talking points

09 Numbers to remember (cheat-sheet)

01 The market

re-forge monetizes the **agent's accumulated context**, not the agent itself, so it captures value whether you frame the market as AI agents or the narrower code-tools slice.

\$52.6B · 2030

TAM, AI agents. 46.3% CAGR from \$7.84B in 2025 (MarketsandMarkets); ~\$50.3B corroboration (Grand View).

\$8.1B · SAM

Serviceable seats, fully penetrated. Bottom-up from developers already on an AI assistant.

\$40M · SOM

Obtainable ARR wedge, ~0.5% of the serviceable base. The multi-year ceiling, not year one.

LAYER	FIGURE	SOURCE / BASIS
TAM · AI agents, 2030	\$52.62B · 46.3% CAGR	MarketsandMarkets
TAM corroboration	~\$50.3B	Grand View Research
TAM (conservative) · AI code tools, 2030	~\$26B	Grand View Research
SAM · serviceable seats, full penetration	~\$8.1B	Bottom-up (below)
SOM · obtainable ARR wedge	~\$40M	~0.5% of seat base

SAM bottom-up: priced as a layer on top of an existing assistant

36.5M professional developers worldwide	SlashData, 2025
× 62% on at least one AI coding assistant / agent	JetBrains State of the Dev Ecosystem, 2025
= ~22.6M serviceable developers	
× \$360 / seat / year (planned Team, \$30/seat/mo)	
= ~\$8.1B SAM at full penetration	

Every developer in the SAM has already adopted an agent, so there is **no cold-start adoption problem at the assistant level**. The job is to attach to a workflow that already exists.

THE ENTRY WEDGE (UNDERWRITE THIS, NOT THE \$40M LEAP)

The first measurable milestone is the **first 1,000 to 3,000 paid seats**, converted out of open-source adoption of the 128 free skills and the Claude Code operating procedure. The plan-of-record is an OSS-to-paid funnel; \$40M is the ceiling of the immediate wedge, not year one.

Why now

- Agents shipped, memory did not. Copilot has 20M+ users, Cursor hit ~\$2B ARR in about three years, yet both stay mostly stateless across sessions and amnesiac across projects. The pain is now acute and universal.
- Memory-as-infrastructure is being funded. Mem0 raised \$24M (seed + Series A, closed Oct 2025) and crossed ~48k GitHub stars. Investors price memory as a category now, not a feature.

- **Multi-tool reality.** Teams use Claude Code and Cursor and Codex and ChatGPT at once. A model-agnostic layer that unifies them is only valuable, and only buildable, now that the sprawl exists.
- **Quality and trust gap.** As agents write more code, the cost of a confident wrong answer rises. Adversarial gates and evidence-on-disk become a procurement requirement, not a nicety.

Sources: MarketsandMarkets AI Agents Market (marketsandmarkets.com/PressReleases/ai-agents.asp) · Cursor \$2B ARR (thenextweb.com) · Mem0 \$24M Series A (mem0.ai/series-a).

02 Competitors & why we win

re-forge is a **layer**. In every category the structural point is the same: these products are where re-forge plugs in, not what it replaces.

A · Single-agent assistants & editors (where we plug in)

PLAYER	WHAT THEY DO
GitHub Copilot	In-editor completion + chat; 20M+ users; Microsoft distribution
Cursor (Anysphere)	AI-native editor; ~\$2B ARR; 1M+ paying, 70% of Fortune 1000
OpenAI Codex	Agentic coding from OpenAI
Amazon Q Developer	AWS-integrated coding assistant

Why we win: a team on Cursor + Copilot has two amnesiac assistants; re-forge is the shared brain over both. Cross-tool by design, memory that compounds **per team** not per vendor, adversarial gates (skeptic / adversary / moderator / evaluator) an autocomplete-first product structurally never ships, and auditable evidence files for regulated buyers.

B · Agent frameworks (construction kits, not a finished product)

PLAYER	WHAT THEY DO
LangGraph	Graph-based agent orchestration
AutoGen (Microsoft)	Multi-agent conversation framework
CrewAI	Role-based multi-agent crews

Why we win: frameworks are a blank canvas; re-forge is a hardened, assembled system that works out of the box. It imports peer-reviewed methods (MAST taxonomy from NeurIPS 2025, ACE from Stanford, DebateCV, Magentic-One, Anthropic's multi-agent research system) and ships built-in adversarial gates plus a file-locked concurrent merge most teams never build correctly. Building re-forge on LangGraph is like building a CRM on a database.

C · LLM / agent observability & evals (read-only telemetry)

PLAYER	WHAT THEY DO
LangSmith	Tracing + eval for LLM apps
Arize Phoenix / Langfuse	Open-source LLM observability + tracing
Helicone	LLM request logging + analytics

Why we win: observability tells you what happened after the fact. re-forge changes what happens **before output ships** by gating it adversarially, then writes lessons back so the same mistake is not repeated. They run side by side; re-forge's evidence files can feed an observability backend.

D · Agent memory & skill infrastructure (the closest neighbors)

PLAYER	WHAT THEY DO
Mem0	Memory layer; ~48k stars; \$24M raised; AWS Agent SDK memory provider
Letta (ex-MemGPT)	Stateful agents with long-term memory
Zep	Memory + knowledge-graph layer for agents

Why we win: this is the only category with real overlap, and even here the shape differs. They are **horizontal, general-purpose** memory primitives (a vector or key-value store a developer integrates). re-forge is **vertical and integrated** for software teams: memory + auto-forged skills + adversarial gates, shipped with 128 skills, 6 specialist teams and a Capability Forge meta-agent. Storing facts is the commodity; **forging team-specific skills, curating lessons, and gating output** is the product. We could even run on top of a primitive like theirs.

03 The moat

No single layer is unbreachable. The defensibility is the **compounding interaction of all three** plus demonstrated research and engineering rigor. A competitor must replicate the data, the agnostic architecture, and the quality system at once, while the incumbent's per-team asset keeps compounding.

Layer 1 · Compounding team-memory data network effect

Every session writes lessons back via the ACE pattern (generation, reflection, curation). Memory accumulates per team and transfers across that team's projects. The more a team uses re-forge, the more valuable it is **to that team specifically**, and that value is data no base model and no competitor possesses.

HONEST COUNTER

A team-scoped network effect is not a classic multi-sided one. It does not make re-forge more valuable to **other** teams.

REBUTTAL

Correct, and that is the **stronger** moat here: a per-team data asset is a per-account switching cost that grows monotonically with use. The Datadog / Snowflake pattern. We claim compounding stickiness, not a viral marketplace effect, which is more defensible and easier to underwrite.

Layer 2 · Model-agnostic & tool-agnostic positioning

re-forge wins regardless of which model or assistant wins, because it is a layer **above** both. If Claude leads, it runs on Claude; if a frontier model leapfrogs, it runs on that; if a team splits across Cursor and Codex, it spans them.

HONEST COUNTER

The shipped product **today** runs only on Claude Code, so "agnostic" is currently roadmap, not reality.

REBUTTAL

Stated plainly: today's shipped product is the Claude Code procedure; the cross-tool monitor is roadmap. **But the hard part already ships.** The cross-tool layer is built on the concurrent-merge primitive (flock + atomic-rename) exercised by the 685-check green CI. The rest is surface integration, not new core infrastructure.

Layer 3 · Switching cost that compounds with use

Layers 1 and 2 combine. A team with months of accumulated memory and a library of auto-forged, personalized skills cannot move without abandoning a custom-fitted asset that took real usage to build. The cost of leaving rises every week.

HONEST COUNTER

Memory could be exportable; a competitor could ship an importer to neutralize lock-in.

REBUTTAL

Raw facts export; the **forged skills, curated lessons, evidence trail and team-specific tuning are an integrated system**, not a flat file. Even a perfect fact export loses the reflection-and-curation structure that makes memory useful. The moat is the compounding **fit**, not the storage format.

THE HONEST SYNTHESIS

The defensibility is the interaction of all three layers plus externally validated rigor: two ICML 2026 SCALE Workshop papers, a 685-check CI, and imported peer-reviewed methods. We do not claim any single unbreachable barrier; we claim a system that is hard to replicate simultaneously while the per-team asset keeps growing.

Pricing (planned)

TIER	PRICE	WHO
Free / OSS	\$0	128 skills + the Claude Code operating procedure. The top of the funnel.
Team	\$30 / seat / mo	Shared team memory, forged skills, audit gates. The SAM unit (\$360/seat/yr).
Enterprise	custom	On-prem memory, SSO, audit controls for regulated buyers.

Funnel: OSS adoption to paid seats. First milestone is 1,000 to 3,000 paid seats, not an immediate leap to the \$40M wedge.

Traction / proof (shipped & verifiable, no fabrication)

Top 5 · 130+

Hackathon Demo Day. Top-5 project, 130+ votes.

685 ✓

Machine checks, green CI on every push.

128 / 6+1

Skills, 6 teams + Forge meta-agent.

2 · ICML

Papers accepted, SCALE Workshop, ICML 2026.

The two ICML 2026 papers (the system researched itself)

Compute-Aware Mixture-of-Agents: Verifier-Gated Adaptive Aggregation under a Fixed Token Budget Shine Gupta · S Akash	Spend a fixed token ceiling optimally; gate aggregation through a verifier. This is the principled version of re-forge's own cost story.
Who Hallucinates Tools, How Often, and What Fixes It? S Akash · Shine Gupta	Quantifies tool hallucination and the fix (tool-registry re-prompting), now fed back into re-forge's agents.

Both runs were orchestrated by re-forge teams, then the findings merged back into the agents' own prompts. The papers are both output of the system and inputs to its next version.

WHY THIS TEAM

The work imports and correctly applies peer-reviewed methods rather than reinventing them, and the results are externally validated by peer review. **S Akash** is a EuroLLVM 2026 speaker with IIT Patna, CERN GSoC and vLLM-contributor credentials. This is a team that ships peer-reviewed systems infrastructure, which is what a defensible memory-and-quality layer requires.

The shipped artifact is a **hardened operating procedure** on Claude Code's subagent primitives, distributed as markdown agent definitions plus shell installers. There is no application server. The "binary" is prompts and protocols; the 685-check harness is the test suite.

Architecture: orchestrator-worker

A lead (research / engineering / security / testing / docs) is the single entry point. It **decomposes** the question into disjoint sub-questions, **dispatches** specialists in parallel (Anthropic's 1 / 2-4 / 10+ scaling rule), and **sequences** the gates. Workers never talk directly; they communicate only through **evidence files on disk**, which eliminates the MAST conversation-corruption failure mode. Six specialist teams (80+ agent files) plus the Forge meta-agent.

Four adversarial gates (mapped to the MAST failure taxonomy, NeurIPS 2025)

GATE	ATTACKS	WHY IT WORKS
Skeptic	the reasoning	Forces each conclusion to re-derive from evidence; counters anchoring.
Adversary	the sources	Treats the corpus as hostile: content farms, benchmark fraud, astroturf. Different job from the skeptic.
Moderator	contradictions	Structured 3-round debate, 5 verdict types, removing leader bias from conflict resolution.
Evaluator	the output	Scores Anthropic's 5-dimension rubric; a strict-dimension miss blocks PASS.

THE PROOF POINT

During a pilot the adversary caught a widely-starred memory project publishing a **96.6% benchmark number that actually measured a vanilla ChromaDB lookup**, not its advertised architecture. The skeptic alone could not catch this: the reasoning was coherent on top of a fraudulent source. That is the architectural argument for separating "attack the argument" from "attack the evidence."

Memory: the ACE loop

Generator (retrospector) extracts 3-7 durable lessons at session close. Reflector applies a durability test ("would this help in 3 months on an unrelated question?"). Curator (scribe) merges into a persistent `MEMORY.md`, deduplicating via grow-and-refine. Memory is **global** so lessons transfer across projects; sessions are **per-project**.

The hard primitive: concurrent merge

Many teams finish near-simultaneously wanting to append to one file. Naive append corrupts. Solution: `flock` (5s acquire) + `timeout --kill-after` (closes a lock-leak found empirically) + write-to-temp-then-atomic mv. Measured: **10 concurrent writers, ~70ms, zero lost writes**. This is the foundation the cross-tool roadmap rests on.

Evidence-as-contract & Eval-Driven Development

Every specialist writes its evidence to disk, and a worker with no file is treated as a worker that did not run (full-activation enforcement). **No code is written until its quality criteria already exist on disk** in `EXPECTED_EVALS.md`. An auditor runs `audit_evidence.py <slug> --strict` to reconstruct why a verdict was reached from files, not transcripts. The difference between "the agent says it's done" and "here is the on-disk proof."

THE ROADMAP, FRAMED HONESTLY

Shipped: the Claude Code procedure, 128 skills, 685-check CI, the Demo Day result, accepted papers. **Roadmap:** an always-on local-first monitor across Claude Code, Cursor, Codex and ChatGPT, plus live multi-developer shared memory. Credible because the hard part (concurrency-safe merge + a working Forge pipeline) is already shipped and tested. Not yet shipped: the cross-tool ingestion surface and multi-tenant sync.

06 Investor objections + rebuttals

1 "Isn't this a feature, not a company?"

Memory looked like a feature until **Mem0 raised \$24M and AWS named it the exclusive memory provider for its Agent SDK**. The market prices memory as infrastructure. re-forged is broader: it bundles memory, skill-forging and an adversarial quality system into a team-scoped layer that compounds per account. Features do not produce a per-team data asset that grows with use; platforms do.

2 "What stops Cursor or Copilot from building this?"

Two structural reasons. **Conflict of interest**: an agnostic layer spanning Cursor and Copilot and Codex cannot be built by any one of them, because each is incentivized to lock memory to its own surface, and a team using two assistants will not trust either walled garden. **Focus**: their roadmaps are model and editor quality; a cross-tool memory-and-audit layer is orthogonal and self-cannibalizing for them. Our neutrality is exactly what an incumbent cannot offer.

3 "You are pre-revenue."

Correct, stated plainly. Pricing is planned, not live. What de-risks the stage is **shipped, verifiable proof**: a top-5 Demo Day result with 130+ votes, 685-check green CI, 128 OSS skills, 6 teams + Forge, two ICML 2026 papers. The funnel is OSS to paid; first milestone 1,000 to 3,000 seats, not the \$40M wedge.

4 "What is your moat against Mem0, Letta and Zep?"

They are horizontal, general-purpose memory primitives. re-forged is a **vertical, integrated layer for software teams** combining memory with auto-forged skills and adversarial gates, and could even run on top of a primitive like theirs. Storing facts is the commodity; forging team-specific skills, curating lessons and gating output is the differentiated product. The concurrent-merge primitive is the infrastructure the cross-tool roadmap rests on.

5 "Why now? Why not two years ago or two years from now?"

Two years ago, agents were not adopted enough for a layer on top to have a market. Today **62% of professional developers use an AI assistant**, multi-tool sprawl is the norm, and memory has just been funded as a category. Two years from now, per-team data assets will already be locked to whoever moved first. The window is now: agents ubiquitous but still amnesiac.

6 "It only works on Claude Code today; the agnostic story is vaporware."

The shipped product is the Claude Code procedure; the cross-tool monitor is roadmap, and we will not pretend otherwise. But the hardest component, the **file-locked concurrent-merge primitive** for safe parallel writes, is already built and exercised by green CI. Extending to Cursor, Codex and ChatGPT is integration on an existing foundation, not net-new core research.

7 "The cross-tool monitor is your headline but isn't shipped. Selling roadmap as traction?"

No, and we separate them explicitly. **Shipped**: the Claude Code procedure, 128 skills, 685-check CI, the Demo Day result, the accepted papers. **Roadmap**: the always-on cross-tool monitor and live shared team memory. We underwrite the company on the shipped product plus the OSS funnel; the monitor is upside, not the base case.

8 "Why is this team the one to build it?"

The work imports and correctly applies peer-reviewed methods (MAST, ACE, DebateCV, Magentic-One, Anthropic's multi-agent system) and the results are externally validated by two ICML 2026 SCALE Workshop papers. **S Akash** is a EuroLLVM 2026 speaker with IIT Patna, CERN GSoC and vLLM-contributor credentials. A team that ships peer-reviewed systems infrastructure, which is exactly what this layer requires.

07 Likely technical questions + answers

Q1 Scalability. Does it fall over as you add teams / agents?

The orchestrator-worker topology scales horizontally: workers are stateless w.r.t. each other and communicate only through disk, so adding specialists adds breadth without $O(n^2)$ chatter. Cross-team parallelism is bounded only by the host's concurrency cap and the memory merge, already proven safe at 10 concurrent writers. The ceiling is the host runtime's limits, not re-forge's design.

Q2 Cost / tokens. Adversarial gates sound expensive.

An explicit, **bounded** cost. A toy /research lands a synthesis in 2-3 minutes for under 50K tokens; a full investigation is 50K-500K. The alternative is a cheap, confident, wrong answer (the fraud the adversary caught). Our Compute-Aware Mixture-of-Agents paper is literally research on spending a fixed token budget optimally. **Cost is a dial, not a fixed tax.**

Q3 Security of evidence files on local disk. What stops tampering?

Evidence lives under the project's `.claude/teams/` and global `~/ .claude/`, governed by OS permissions, the same trust boundary as the source tree. The guarantee is **detection, not DRM**: the audit script verifies every dispatched specialist has its file and that a PASS is backed by a reconciled checklist. Merges are atomic, so a concurrent writer cannot corrupt the record mid-write.

Q4 How does memory avoid context bloat?

Three mechanisms. (1) The reflector's **durability test** discards session trivia at generation; only 3-7 transferable lessons survive. (2) The curator **deduplicates** via grow-and-refine and marks stale entries. (3) Hook A routes long-tail overflow to topic files with reference pointers, keeping the hot `MEMORY.md` small. Memory is a curated playbook, not an append-only log.

Q5 What happens on a model change?

Almost nothing, by design. The procedure is markdown + shell, decoupled from weights. Agents declare a preferred model with graceful fallback to the plan's strongest available; swap the model and the contract (gates, evidence, memory merge) is identical. That is the entire point of being a procedure, not a model.

Q6 Data privacy of the cross-tool monitor (roadmap).

Designed **local-first**: it observes session activity on the developer's machine and the shared store is the team's own (Enterprise offers on-prem memory + SSO). Nothing requires shipping source or transcripts to a third party for the core loop. Stated honestly: this is a design commitment, not yet a shipped, audited control.

Q7 Latency.

Two regimes. Main-thread mode fans workers out in parallel, so wall-clock is roughly the slowest worker plus the serial gate chain (a toy question is 2-3 minutes). Subagent mode degrades to serial (correct but slower). The memory merge adds **~70ms even at 10 concurrent writers**, negligible. Latency is dominated by investigation depth, which the user controls.

Q8 How are skills forged?

Five stages: **gap** (diff the capability set against a taxonomy, often seeded by a recurring memory lesson), **scout** (check the MCP Registry to reuse before building), **draft** (wrap the official skill-creator / mcp-builder), **test** (smoke case), **promote** (install + attach helpful/harmful counters). Proven end-to-end: the Forge authored the `hn-search` skill in a smoke test. Forged skills are then drift-checked by the same 685-check harness.

08 Demo talking points

The demo is one story: **"looks done" is not "verified."** Show the gap, give credit where it is due, never cry wolf.

1 Frame the layer, not the model

One prompt becomes a coordinated org of specialist agents. re-forged sits above the assistant.

Say: *"This is not another copilot. It is the layer that makes whichever copilot you use remember and prove its work."*

2 Give the plain agent credit

A plain agent built secure JWT auth and did a genuinely good job: sound token core (Argon2id, refresh rotation, reuse-detection, revocation) that passed all 19 of its own tests.

Say: *"Green tests, real work. We give credit where it's due. Watch what the tests missed."*

3 The adversary finds the hole the suite missed

The security team audited that passing code and found broken object-level authorization (IDOR): any logged-in user could read and DELETE every other user's data. Reproduced cross-user against the running app.

Say: *"Nineteen green tests. One HIGH-severity hole they could never have caught. That gap is the product."*

4 It refuses to inflate

Honest verdict: 1 HIGH + 5 MEDIUM, marked ADVISORY. The adversary refused to inflate demo-convenience findings.

Say: *"It doesn't cry wolf. An honest gate is the only one a regulated buyer will trust."*

5 Close on the compounding loop

The lesson is written to memory; the next session starts smarter. Point at the green 685-check CI and the two ICML papers the system produced on itself.

Say: *"Every session makes the next one smarter. That is the asset that compounds, and the one no single model can own."*

09 Numbers to remember

MEMORIZE THESE • DO NOT FUMBLE A FIGURE ON STAGE

\$52.6B TAM, AI agents 2030, 46.3% CAGR (MarketsandMarkets)

~\$26B Conservative TAM, AI code tools 2030 (Grand View)

~\$8.1B SAM: 22.6M serviceable devs × \$360/seat/yr

~\$40M SOM, ~0.5% of seat base (multi-year ceiling)

36.5M Professional developers worldwide (SlashData)

62% On at least one AI assistant (JetBrains)

1k-3k First paid-seat milestone (the real near-term target)

\$0 / \$30 Free OSS / Team per seat per month; Enterprise custom

685 Machine checks, green CI on every push

128 / 6+1	Skills / specialist teams + Forge meta-agent
Top 5 · 130+	Demo Day placement and votes
2	ICML 2026 SCALE Workshop papers accepted
~70ms	Memory merge at 10 concurrent writers, zero lost writes
<50K	Tokens for a toy research synthesis (2-3 min)
\$24M	Mem0 raise; AWS exclusive memory provider (the comp)
~\$2B	Cursor ARR; Copilot 20M+ users (the amnesiacs)